

Semaine 4 | Module C**CSS — Cascade, modèle de boîte & Flexbox**

Anne Terrier — Développement web, classe passerelle

Cours 3h

TP 4h

Fil rouge MiniForum

Objectifs de la semaine

- Brancher une feuille de style externe et écrire des règles CSS valides
- Choisir le sélecteur adapté à ce que l'on veut atteindre
- Savoir quelle règle l'emporte en cas de conflit : cascade, spécificité, héritage
- Comprendre le modèle de boîte et le rôle décisif de `box-sizing`
- Construire une mise en page unidimensionnelle avec Flexbox
- Réagir à l'état d'un élément avec les pseudo-classes (`:hover`, `:focus`, `:nth-child`)
- Inspecter et modifier des styles dans les outils de développement

Où en sommes-nous ?

Trois semaines de HTML : le MiniForum a une structure complète, sémantique, validée et navigable au clavier — et il est laid. C'était voulu.

À partir de cette semaine, on ne touche presque plus au HTML. Tout le travail se fait dans un fichier séparé, `style.css`, qui vient se poser **par-dessus** la structure existante. C'est le moment où l'on mesure ce qu'un HTML bien écrit rapporte : une page structurée n'importe comment est un enfer à styler.

Cette semaine couvre les fondations : sélectionner, comprendre qui gagne, mesurer les boîtes, aligner en une dimension. La semaine 5 ajoute la deuxième dimension (Grid), l'adaptation aux écrans (*responsive*) et les variables CSS.

1 Le rôle du CSS**Définition — CSS**

CSS (*Cascading Style Sheets*, « feuilles de style en cascade ») est le langage qui décrit la **présentation** d'un document HTML : couleurs, polices, tailles, espacements, positions.

Le HTML dit *ce que sont* les éléments. Le CSS dit *à quoi ils ressemblent*.

Pourquoi séparer structure et présentation

Les deux langages pourraient être mélangés — et l'étaient, dans les années 1990. On les sépare pour trois raisons très concrètes :

- **Une seule feuille pour tout le site.** Changer la couleur des boutons du MiniForum se fait à un endroit, pas dans quatre fichiers HTML.
- **Le HTML reste lisible.** Un fichier où la structure disparaît sous les instructions de mise en forme n'est plus maintenable.
- **Une même structure, plusieurs présentations.** Écran large, téléphone, impression : le HTML ne change pas, seul le CSS s'adapte. C'est tout le sujet de la semaine 5.

Anatomie d'une règle

```

1 h1 {
2   color: #1D4E89;
3   font-size: 32px;
4 }
```

Extrait 1 – Une règle CSS et son vocabulaire

Terme	Ce que c'est
Sélecteur	h1 — désigne les éléments visés
Bloc de déclarations	Ce qui se trouve entre les accolades
Déclaration	color: #1D4E89; — une propriété et sa valeur
Propriété	color — ce que l'on modifie
Valeur	#1D4E89 — comment on le modifie

Le point-virgule et les deux-points

Une déclaration s'écrit propriété : valeur ;. Le **deux-points** sépare les deux, le **point-virgule** termine la déclaration.

Un point-virgule oublié invalide la déclaration suivante — et souvent elle seule, ce qui rend l'erreur difficile à repérer. Le CSS ne signale rien : il ignore silencieusement ce qu'il ne comprend pas et continue. C'est la différence majeure avec les langages que tu verras en POO.

Trois façons de brancher du CSS

```

1 <!-- 1. Attribut style : sur un seul element -->
2 <p style="color: red;">Alerte</p>
3
4 <!-- 2. Balise <style> : dans le <head> de cette page -->
5 <style>
6   p { color: red; }
7 </style>
8
9 <!-- 3. Fichier externe : dans le <head>, valable pour tout le site -->
10 <link rel="stylesheet" href="style.css">
```

Extrait 2 – Les trois emplacements possibles

Méthode	Portée et usage	Dans ce cours
Attribut style	Un seul élément. Illisible et impossible à réutiliser	à éviter
Balise <style>	Une seule page. Acceptable pour un essai rapide	essais seulement
Fichier .css	Tout le site, mis en cache par le navigateur	la règle

L'attribut style est proscrit dans le MiniForum

Il ramène la présentation dans le HTML, exactement ce que la séparation cherche à éviter. Pire : il possède une spécificité si élevée (section 3) qu'aucune règle de la feuille de style ne peut le contredire, ce qui produit des heures de « pourquoi mon CSS ne s'applique pas ».

Une seule exception, plus tard dans l'année : quand JavaScript modifie l'apparence d'un élément à la volée (semaine 7).

Le <link> va dans le <head>

<link rel="stylesheet" href="style.css"> se place dans le <head>, avant tout contenu. Placé plus bas, le navigateur affiche brièvement la page non stylée avant de la repeindre — ce clignotement porte un nom, le *flash of unstyled content*, et il est visible.

Si rien ne change après l'ajout du <link> : vérifie le chemin du fichier (semaine 2), et recharge en forçant le cache avec Ctrl + Maj + R.

Commentaires et lisibilité

```
1 /* Barre de navigation ----- */
2
3 nav {
4     background-color: #1A1A1A;    /* fond sombre */
5 }
```

Extrait 3 – Le seul commentaire du CSS

Le CSS ne connaît qu'une forme de commentaire, /*...*/, sur une ou plusieurs lignes. Le // de JavaScript n'existe pas ici : écrit dans un fichier CSS, il casse la règle qui suit.

2 Les sélecteurs

Un sélecteur répond à une question : *à quels éléments cette règle s'applique-t-elle ?*

Les sélecteurs de base

```
1 /* Type : tous les <p> du site */
2 p { line-height: 1.6; }
3
4 /* Classe : tous les elements avec class="carte-sujet" */
5 .carte-sujet { background-color: #FFFFFF; }
6
7 /* Identifiant : l'unique element avec id="menu-principal" */
8 #menu-principal { background-color: #1A1A1A; }
9
10 /* Universel : absolument tout */
11 * { box-sizing: border-box; }
```

Extrait 4 – Type, classe, identifiant, universel

Sélecteur	Écriture	Vise
Type	p, h2, nav	tous les éléments de ce nom
Classe	.carte-sujet	tous les éléments portant cette class
Identifiant	#menu-principal	l'unique élément portant cet id
Universel	*	tous les éléments

Côté HTML, la classe s'ajoute avec l'attribut class, et un élément peut en porter plusieurs, séparées par une espace :

```
1 <article class="carte-sujet carte-epinglee">
2   <h3>Comment installer Node ?</h3>
3 </article>
```

Extrait 5 – Un élément, deux classes

Classe ou identifiant ? La classe, presque toujours

Les deux fonctionnent pour styler. Mais l'id doit rester **unique dans la page** (semaine 2), ce qui interdit de réutiliser la règle ailleurs ; et sa spécificité écrasante rend toute correction ultérieure pénible.

Dans ce cours : l'id sert au <label for>, aux ancres et — dès la semaine 7 — à JavaScript. Le **style passe par les classes**.

Les combinateurs

On peut viser un élément selon sa position dans l'arbre du document.

```
1 /* Descendant : tous les <a> situes quelque part dans <nav> */
2 nav a { color: white; }
3
4 /* Enfant direct : les <li> enfants immédiats de <ul> */
5 ul > li { margin-bottom: 8px; }
6
7 /* Frere adjacent : le <p> qui suit immédiatement un <h2> */
8 h2 + p { font-weight: bold; }
9
10 /* Groupe : la meme regle pour plusieurs selecteurs */
11 h1, h2, h3 { font-family: Georgia, serif; }
```

Extrait 6 – Quatre façons de combiner

Combinateur	Exemple	Lecture
Espace	nav a	un <a> quelque part dans un <nav>
>	ul > li	un enfant direct d'un
+	h2 + p	le <p> juste après un <h2>
Virgule	h1, h2	les <h1> et les <h2>

L'espace et la virgule ne sont pas interchangeables

nav a vise les liens **à l'intérieur** de la navigation.

nav, a vise la navigation **et** tous les liens de la page, y compris ceux du pied de page.

Une virgule oubliée ou ajoutée par erreur change complètement la portée d'une règle, sans provoquer la moindre erreur. Relis toujours tes sélecteurs à voix haute.

Nommer ses classes**Nommer par le rôle, jamais par l'apparence**

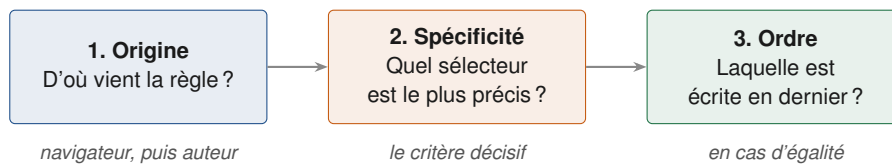
.carte-sujet décrit **ce qu'est** l'élément. .boite-bleue décrit ce à quoi il ressemble aujourd'hui.

Le jour où les cartes passent au gris, la première classe reste juste et la seconde devient un mensonge que personne n'ose corriger. Convention du cours : en minuscules, mots séparés par un tiret, sans accent — .carte-sujet, .bouton-primaire, .message-erreur.

3 La cascade, la spécificité, l'héritage

C'est le cœur du CSS, et la source de la quasi-totalité des « ça ne marche pas » de la semaine. Deux règles peuvent viser le même élément et dire le contraire. Il faut donc un arbitrage.

Trois questions, dans cet ordre



Le navigateur applique ces trois critères **dans l'ordre** et s'arrête dès qu'un départage a lieu. En pratique, dans ce cours, tout se joue sur les critères 2 et 3.

Calculer une spécificité

La spécificité se lit comme un score à trois chiffres : **(identifiants, classes, types)**. On compte, dans le sélecteur, combien il y a de chaque, puis on compare de gauche à droite.

Sélecteur	Score	Lecture
*	0-0-0	le sélecteur universel ne compte pas
p	0-0-1	un type
nav a	0-0-2	deux types
.carte-sujet	0-1-0	une classe
a:hover	0-1-1	une pseudo-classe compte comme une classe
nav a.aktif	0-1-2	deux types et une classe
#menu-principal	1-0-0	un identifiant
style="..."	<i>hors concours</i>	bat tout sélecteur

Exemple — qui gagne ?

```

nav a      { color: white; } /* 0-0-2 */
.lien-aktif { color: yellow; } /* 0-1-0 */
  
```

Pour un lien `<a class="lien-aktif">` placé dans la navigation, les deux règles s'appliquent. On compare de gauche à droite : les identifiants sont à égalité (0), puis **1 classe l'emporte sur 0**. Le lien est **jaune**.

Le nombre de types (2 contre 0) n'est jamais examiné : la comparaison s'arrête à la première colonne qui départage. Cent sélecteurs de type ne battront jamais une seule classe.

À spécificité égale, le dernier écrit gagne

```

.bouton { background-color: blue; }
.bouton { background-color: green; }
  
```

Même score (0-1-0) : c'est l'ordre du fichier qui tranche, le bouton est **vert**. C'est aussi pour cette raison qu'une feuille de style se lit du général vers le particulier : les règles générales en haut, les exceptions en bas.

!important : la sortie de secours à ne pas prendre

!important est interdit dans le MiniForum

Ajouter !important à une déclaration la fait passer devant tout le reste, spécificité comprise :

```
color: red !important;
```

C'est efficace, et c'est exactement le problème : la règle devient impossible à contredire autrement qu'avec un second !important, puis un troisième. Une feuille de style qui en contient est une feuille dont plus personne ne contrôle le résultat.

Quand tu es tenté d'en écrire un, c'est presque toujours qu'une règle plus spécifique existe ailleurs — souvent un #id ou un attribut style. Trouve-la dans les outils de développement et corrige la cause.

L'héritage

Définition — Héritage

Certaines propriétés se transmettent automatiquement d'un élément à ses descendants. Une couleur définie sur <body> descend dans tous les paragraphes de la page, sans qu'aucune règle ne les vise.

S'hérite

color, font-family, font-size, font-weight, line-height, text-align

Ne s'hérite pas

margin, padding, border, background, width, height, display

La logique est simple : ce qui concerne le **texte** s'hérite, ce qui concerne la **boîte** ne s'hérite pas. Une bordure héritée par tous les descendants serait absurde.

```
1 body {
2   font-family: Helvetica, Arial, sans-serif;
3   font-size: 16px;
4   line-height: 1.6;
5   color: #1A1A1A;
6 }
```

Extrait 7 – Poser la typographie une seule fois

Le réglage le plus rentable de la semaine

Ces quatre déclarations sur body habillent la totalité du texte du site. Toute règle plus précise écrite ensuite viendra simplement les surcharger là où c'est nécessaire.

Un line-height de 1.6 (sans unité) est l'un des rares réglages qui améliore la lisibilité d'un site en une ligne. Sans unité, il se recalcule correctement pour chaque taille de police héritée.

Les champs de formulaire n'héritent pas de la police

<input>, <textarea>, <select> et <button> conservent la police par défaut du système, même si body en impose une autre. Il faut la leur redonner explicitement :

```
input, textarea, select, button { font-family: inherit; }
```

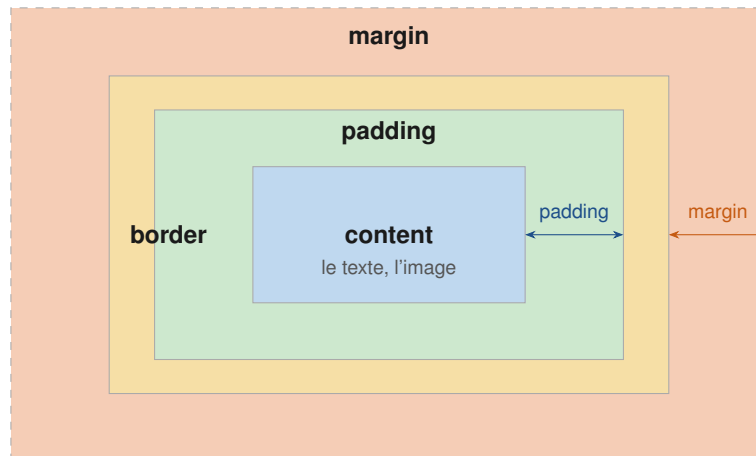
Sans cette ligne, les formulaires du MiniForum resteront visuellement étrangers au reste du site. La valeur inherit force une propriété à reprendre celle du parent.

4 Le modèle de boîte

Définition — Modèle de boîte

Chaque élément d'une page est une **boîte rectangulaire**, composée de quatre couches emboîtées : le contenu, l'espace intérieur (padding), la bordure (border) et l'espace extérieur (margin).

Les quatre couches



Couche	Rôle
content	Le contenu lui-même : texte, image. Ses dimensions sont width et height
padding	L'espace intérieur , entre le contenu et la bordure. Il prend la couleur de fond
border	Le trait qui entoure la boîte : épaisseur, style, couleur
margin	L'espace extérieur , qui écarte la boîte de ses voisines. Toujours transparent

padding ou margin ? La question du fond

Les deux créent de l'espace, mais le padding est **dans** la boîte : il prend la couleur de fond et agrandit la zone cliquable. Le margin est **hors** de la boîte : il reste transparent.

Pour aérer le texte à l'intérieur d'une carte de sujet : padding. Pour écarter deux cartes l'une de l'autre : margin — ou mieux, gap si le parent est en Flexbox (section 5).

Les raccourcis

```

1 padding: 16px; /* les 4 cotes */
2 padding: 16px 24px; /* haut-bas / gauche-droite */
3 padding: 8px 16px 24px; /* haut / gauche-droite / bas */
4 padding: 8px 16px 24px 32px; /* haut / droite / bas / gauche */
5
6 border: 1px solid #CCCCCC; /* epaisseur / style / couleur */
7 border-radius: 8px; /* coins arrondis */
8
9 margin: 0 auto; /* centrer horizontalement */

```

Extrait 8 – De une à quatre valeurs

Quatre valeurs : dans le sens des aiguilles d'une montre

Haut, droite, bas, gauche — on part du haut et on tourne. La même règle vaut pour `margin` et pour `padding`.

Avec deux valeurs, la première est le couple haut-bas, la seconde le couple gauche-droite. C'est de très loin la forme la plus utilisée.

`margin: 0 auto` pour centrer un bloc

Un bloc de largeur définie se centre horizontalement avec `auto` sur les marges gauche et droite : le navigateur répartit l'espace restant à parts égales.

```
main { max-width: 900px; margin: 0 auto; }
```

C'est la recette du contenu centré dans une page large, et elle sert dans tous les sites. Elle ne fonctionne que sur un élément de type bloc dont la largeur est bornée.

`box-sizing` : la propriété qui change tout

Par défaut, `width` désigne la largeur du **contenu seul**. Le `padding` et la `border` viennent s'y ajouter.

```
1 .carte {
2   width: 300px;
3   padding: 20px;
4   border: 2px solid black;
5 }
6 /* Largeur réellement occupée :
7    300 + 20 + 20 + 2 + 2 = 344 px */
```

Extrait 9 – Le calcul par défaut, et le piège

Deux cartes de 50 % ne tiennent pas côte à côte

C'est le grand classique de la semaine : deux blocs à `width: 50%` avec un `padding` passent à la ligne, parce que leur largeur réelle dépasse 50 %.

On peut soustraire le `padding` à la main. Personne ne le fait — on change le mode de calcul.

```
1 * {
2   box-sizing: border-box;
3 }
```

Extrait 10 – La première règle de toute feuille de style

Valeur	Ce que <code>width</code> désigne alors
<code>content-box</code>	Le contenu seul — le <code>padding</code> et la <code>border</code> s'ajoutent (<i>défaut</i>)
<code>border-box</code>	La boîte bordure comprise — le contenu se réduit d'autant

À écrire en haut de `style.css`, une fois pour toutes

Avec `border-box`, une carte à `width: 300px` occupe exactement 300 pixels, quel que soit son `padding`. Les largeurs deviennent prévisibles, et les mises en page cessent de se casser pour deux pixels.

La règle universelle `*{ box-sizing: border-box; }` figure en première ligne de la quasi-totalité des feuilles de style professionnelles. Écris-la, et n'y pense plus de l'année.

La fusion des marges verticales

Deux marges verticales qui se touchent n'en font qu'une

Un paragraphe avec `margin-bottom: 30px` suivi d'un titre avec `margin-top: 20px` ne produit pas 50 pixels d'écart, mais **30** : la plus grande des deux marges absorbe l'autre.

Ce comportement, appelé *fusion des marges* (*margin collapsing*), ne concerne que les marges **verticales** d'éléments de type bloc. Les marges horizontales, elles, s'additionnent normalement.

Il n'y a rien à corriger : c'est voulu, et c'est ce qui évite des espacements doublés partout. Il faut simplement le savoir avant de passer vingt minutes à chercher les pixels manquants. À noter : la fusion ne se produit ni dans un conteneur Flexbox, ni en Grid — une raison de plus d'utiliser gap.

display : le type de boîte

Valeur	Comportement
block	Occupe toute la largeur disponible, se place sur une nouvelle ligne. width, height et les marges verticales s'appliquent
inline	Reste dans le fil du texte. width, height et les marges verticales sont ignorées
inline-block	Reste dans le fil du texte, mais accepte les dimensions et les marges
flex	Fait de l'élément un conteneur Flexbox (section 5)
none	L'élément disparaît complètement, sans laisser d'espace

Pourquoi la hauteur d'un lien ne change pas

Un `<a>` est `inline` par défaut : lui donner une `height` ou un `margin-top` n'a aucun effet visible. C'est le motif d'étonnement le plus fréquent du TP.

Pour qu'un lien devienne un bouton cliquable de bonne taille, il faut d'abord le sortir du mode `inline` : `display: inline-block` — ou en faire un élément d'un conteneur Flexbox, ce qui produit le même résultat.

display: none cache aussi aux lecteurs d'écran

Un élément en `display: none` n'est pas seulement invisible : il est **retiré de l'arbre d'accessibilité** et devient inatteignable au clavier.

C'est le bon choix pour un contenu réellement hors sujet. C'en est un très mauvais pour du texte que l'on veut réserver aux lecteurs d'écran, ou pour un menu que l'on croit simplement « replié ». On y reviendra en semaine 5 avec le menu mobile.

Couleurs, polices et unités

```

1 color: #1D4E89;                /* hexadecimal : le plus courant */
2 color: rgb(29, 78, 137);        /* rouge, vert, bleu de 0 à 255 */
3 color: rgba(29, 78, 137, 0.5); /* avec transparence */
4 color: darkslateblue;           /* nom predefini : pour tester */

```

Extrait 11 – Écrire une couleur

La notation hexadécimale se lit par paires : `#RRGGBB`, deux caractères par composante, de 00 à FF. La forme courte `#FFF` équivaut à `#FFFFFF`.

```

1 font-family: Helvetica, Arial, sans-serif;

```

Extrait 12 – Une pile de polices

Le navigateur essaie les polices **de gauche à droite** et retient la première disponible sur la machine. Le dernier terme est toujours une famille générique — `sans-serif`, `serif` ou `monospace` — qui garantit un repli correct.

Unité	Signification
px	Pixel. Valeur fixe, simple à raisonner : l'unité de cette semaine
%	Pourcentage d'une dimension du parent
rem, em	Relatives à une taille de police. Semaine 5
vw, vh	Pourcentage de la largeur ou de la hauteur de la fenêtre. Semaine 5

Cette semaine, tout en pixels

Les unités relatives sont indispensables pour un site vraiment adaptable, et c'est le sujet de la semaine prochaine. Pour l'instant, raisonner en pixels évite d'ajouter une difficulté à celles de la cascade et du modèle de boîte.

Une seule exception, déjà rencontrée : `line-height` s'écrit **sans unité**.

5 Flexbox

Le problème que Flexbox résout

Aligner trois blocs côte à côte, de hauteur égale, avec un espacement régulier et un centrage vertical : cela a occupé les développeurs web pendant quinze ans, à coups de `float` et de bricolages. Flexbox règle la question en trois déclarations.

Définition — Flexbox

Flexbox (*Flexible Box Layout*) est un mode de mise en page **unidimensionnel** : il distribue des éléments le long d'une seule direction, en ligne **ou** en colonne, et gère l'espace restant à leur place.

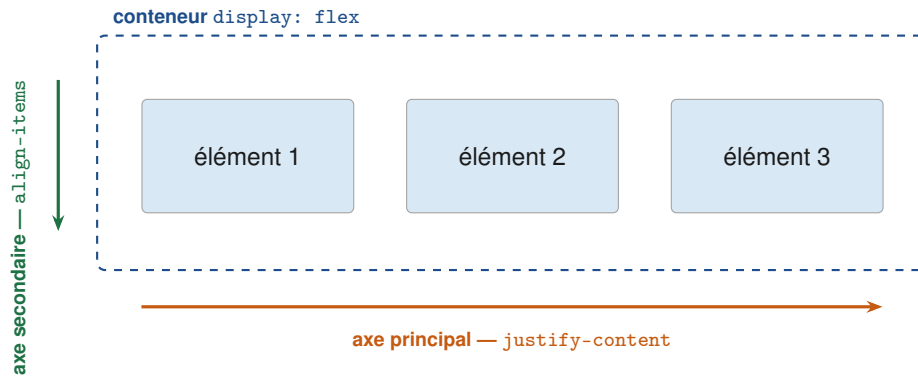
Pour une mise en page à deux dimensions — lignes **et** colonnes simultanément — l'outil est CSS Grid, vu en semaine 5.

Conteneur, éléments, et deux axes

Flexbox se déclare sur le **parent**. Les enfants directs deviennent automatiquement des **éléments flexibles**.

```
1 nav ul {
2   display: flex;
3   justify-content: space-between;
4   align-items: center;
5 }
```

Extrait 13 – Trois lignes suffisent



La clé de Flexbox : deux axes, deux propriétés

`justify-content` agit sur l'**axe principal**. `align-items` agit sur l'**axe secondaire**, perpendiculaire.

Par défaut, l'axe principal est **horizontal** : `justify-content` distribue de gauche à droite, `align-items` aligne verticalement.

Avec `flex-direction: column`, les deux s'inversent : `justify-content` agit alors verticalement. Presque toutes les confusions autour de Flexbox viennent de là. La question à se poser n'est jamais « horizontal ou vertical ? » mais « principal ou secondaire ? ».

Les propriétés du conteneur

Propriété	Valeurs utiles
<code>display</code>	<code>flex</code> — active le mode sur le conteneur
<code>flex-direction</code>	<code>row</code> (défaut), <code>column</code> , <code>row-reverse</code> , <code>column-reverse</code>
<code>justify-content</code>	<code>flex-start</code> (défaut), <code>center</code> , <code>flex-end</code> , <code>space-between</code> , <code>space-around</code> , <code>space-evenly</code>
<code>align-items</code>	<code>stretch</code> (défaut), <code>center</code> , <code>flex-start</code> , <code>flex-end</code> , <code>baseline</code>
<code>flex-wrap</code>	<code>nowrap</code> (défaut), <code>wrap</code> — autorise le passage à la ligne
<code>gap</code>	Espace entre les éléments : <code>gap: 16px</code>

gap plutôt que margin

Avant `gap`, espacer des éléments imposait une marge sur chacun, puis de retirer celle du dernier avec un sélecteur du genre `:last-child`. `gap` pose l'espace **entre** les éléments et nulle part ailleurs, en une déclaration.

Règle du cours : dans un conteneur Flexbox, l'espacement se fait avec `gap`.

`align-items: stretch`, le comportement par défaut

Sans instruction contraire, les éléments d'un conteneur Flexbox s'étirent pour atteindre la hauteur du plus grand. C'est ce qui donne gratuitement des cartes de sujet de hauteur égale, alors que leurs textes ont des longueurs différentes.

Poser `align-items: center` annule cet étirement : les éléments reprennent leur hauteur naturelle et se centrent. Utile dans une barre de navigation, indésirable dans une grille de cartes.

Les propriétés des éléments

Propriété	Effet sur un élément
flex-grow	Part de l'espace libre qu'il absorbe. 0 par défaut
flex-shrink	Sa capacité à se réduire si la place manque. 1 par défaut
flex-basis	Sa taille de départ avant répartition
flex	Raccourci des trois : flex: 1 est la forme courante
align-self	Aligne cet élément seul sur l'axe secondaire
order	Change l'ordre d'affichage sans toucher au HTML

```

1 .barre-recherche {
2   display: flex;
3   gap: 8px;
4 }
5
6 .barre-recherche input {
7   flex: 1;           /* prend tout l'espace disponible */
8 }
9
10 .barre-recherche button {
11   flex: 0;           /* garde sa taille naturelle */
12 }

```

Extrait 14 – Une barre de recherche qui occupe la place restante

order ne réordonne que pour les yeux

order modifie l'ordre **visuel** des éléments. Il ne modifie ni l'ordre du code, ni celui de la tabulation, ni celui que lit un lecteur d'écran.

Un site où l'ordre visuel et l'ordre du clavier divergent devient déroutant, voire inutilisable (rappel du test des trente secondes, semaine 3). Si l'ordre logique doit changer, change le **HTML**.

Deux recettes à connaître par cœur

```

1 .entete {
2   display: flex;
3   justify-content: space-between; /* ecarte les extremes */
4   align-items: center;           /* centre verticalement */
5   padding: 12px 24px;
6 }
7
8 .entete nav ul {
9   display: flex;
10  gap: 20px;
11  list-style: none; /* enleve les puces */
12  margin: 0;
13  padding: 0;
14 }

```

Extrait 15 – Barre de navigation : logo à gauche, menu à droite

```

1 .liste-sujets {
2   display: flex;
3   flex-wrap: wrap; /* passage a la ligne autorise */
4   gap: 16px;
5 }

```

```

6
7 .carte-sujet {
8   flex: 1 1 280px;      /* grandit, retrecit, part de 280px */
9   padding: 16px;
10  border: 1px solid #DDDDDD;
11  border-radius: 8px;
12  background-color: #FFFFFF;
13 }

```

Extrait 16 – Liste de cartes qui passent à la ligne

flex: 1 1 280px est déjà presque du responsive

Les cartes visent 280 pixels, s'étirent pour remplir la ligne, et passent à la ligne suivante dès qu'elles ne tiennent plus. Réduis la fenêtre : la mise en page s'adapte sans qu'aucune *media query* ait été écrite.

C'est un avant-goût de la semaine 5 — et la preuve qu'un bon choix de mise en page évite beaucoup de règles conditionnelles.

list-style: none enlève la puce, pas la sémantique

Un menu de navigation reste un `<ul>` de liens (semaine 2), même quand les puces sont retirées : le lecteur d'écran annonce toujours « liste de cinq éléments ».

Retirer la puce est un choix de présentation ; remplacer le `<ul>` par une suite de `<div>` serait une perte d'information.

6 Les pseudo-classes

Définition — Pseudo-classe

Une **pseudo-classe** cible un élément selon son **état** ou sa **position**, et non selon ce qu'il est. Elle s'écrit après le sélecteur, précédée de deux-points : `a:hover`.

Pseudo-classe	S'applique quand
<code>:hover</code>	Le pointeur survole l'élément
<code>:focus</code>	L'élément a le focus clavier
<code>:focus-visible</code>	L'élément a le focus et le navigateur juge utile de le montrer
<code>:active</code>	L'élément est en cours de clic
<code>:first-child</code> / <code>:last-child</code>	C'est le premier / le dernier enfant de son parent
<code>:nth-child(n)</code>	C'est le n ^e enfant. Accepte even, odd, 2n+1
<code>:checked</code>	Une case ou un bouton radio est coché
<code>:invalid</code>	Un champ ne respecte pas sa contrainte de validation

```

1 nav a:hover {
2   text-decoration: underline;
3 }
4
5 nav a:focus-visible {
6   outline: 2px solid #F5C400;
7   outline-offset: 2px;
8 }

```

```

9
10 .liste-messages li:nth-child(even) {
11     background-color: #F7F7F7;    /* une ligne sur deux */
12 }

```

Extrait 17 – Retour visuel sur les liens et lignes alternées

:hover seul exclut la moitié des utilisateurs

Le survol n'existe pas sur un écran tactile, et n'existe pas non plus pour qui navigue au clavier. Un menu dont les entrées ne réagissent qu'au `:hover` ne donne aucun retour à ces personnes.

Règle du cours : chaque fois que tu écris un `:hover`, écris le `:focus-visible` correspondant. C'est la suite directe du test au clavier de la semaine 3.

Rappel — ne jamais écrire `outline: none`

On trouve cette ligne dans une grande partie du CSS copié en ligne : elle supprime le cadre de focus et rend le site inutilisable au clavier.

Si le cadre par défaut ne convient pas, on le **remplace** par un contour visible, comme ci-dessus. On ne l'efface pas.

:nth-child compte les frères, pas les éléments visés

`li:nth-child(even)` signifie « un `<li>` qui est le 2^e, 4^e, 6^e... enfant de son parent ». Si le parent contient d'autres balises intercalées, le comptage les inclut — source d'un décalage inattendu.

7 Inspecter avec les outils de développement

Les outils de développement (F12) sont l'outil de débogage principal du CSS. Sans eux, comprendre pourquoi une règle ne s'applique pas relève de la devinette.

Panneau	Ce qu'il montre
<i>Elements</i>	L'arbre HTML tel que le navigateur l'a construit
<i>Styles</i>	Toutes les règles qui visent l'élément sélectionné, dans l'ordre de priorité
<i>Computed</i>	La valeur finale de chaque propriété, après arbitrage
Schéma du modèle de boîte	Les valeurs réelles de <code>margin</code> , <code>border</code> , <code>padding</code> et du contenu

La déclaration barrée est la réponse à ta question

Dans le panneau *Styles*, une déclaration **barrée** a perdu l'arbitrage. Le navigateur affiche juste au-dessus la règle qui l'a emporté, avec le fichier et le numéro de ligne.

« Mon CSS ne s'applique pas » se résout presque toujours en trois secondes ici. Les trois causes, par ordre de fréquence : une règle plus spécifique ailleurs, un sélecteur qui ne vise pas l'élément qu'on croit, et une faute de frappe dans le nom de la propriété — dans ce dernier cas, la déclaration apparaît en grisé avec un avertissement.

Modifier dans les DevTools ne sauvegarde rien

Les valeurs se modifient directement dans le panneau *Styles*, avec un aperçu immédiat : c'est la meilleure façon de chercher la bonne valeur d'un padding ou d'une couleur.

Mais tout est perdu au rechargement. Une fois la valeur trouvée, il faut la **reporter dans** `style.css`. Beaucoup de travail a disparu faute de ce réflexe.

8 Organiser sa feuille de style

Une feuille de style qui grossit sans plan devient vite impossible à modifier. Trois habitudes suffisent pour l'éviter.

- **Du général au particulier.** La règle universelle et les styles de `body` en haut, puis les grandes zones (en-tête, contenu, pied de page), puis les composants (cartes, formulaires, boutons). Cet ordre exploite la cascade au lieu de la subir.
- **Des commentaires de section.** Une ligne de séparation avant chaque bloc suffit à retrouver son chemin dans trois cents lignes.
- **Un composant, un endroit.** Tout ce qui concerne la carte de sujet se trouve au même endroit du fichier, pas éparpillé en cinq points.

```

1  /* === 1. Reglages generaux ===== */
2  * { box-sizing: border-box; }
3  body { /* police, couleur, interlignage */ }
4
5  /* === 2. En-tete et navigation ===== */
6
7  /* === 3. Contenu principal ===== */
8
9  /* === 4. Cartes de sujet ===== */
10
11 /* === 5. Formulaires ===== */
12
13 /* === 6. Pied de page ===== */

```

Extrait 18 – Le squelette attendu pour le MiniForum

Pas de CSS copié sans être compris

Un extrait trouvé en ligne peut faire exactement ce que tu cherches — et embarquer un outline : none, un !important ou un float dont tu ne mesures pas les conséquences.

La règle du cours vaut ici comme ailleurs : tout code repris doit pouvoir être expliqué ligne par ligne. Et un CSS écrit par un assistant IA est de toute façon interdit dans les travaux remis.

9 Lexique de la semaine

Terme	Définition
Règle	Un sélecteur et son bloc de déclarations
Déclaration	Un couple propriété / valeur, terminé par un point-virgule
Sélecteur	Expression désignant les éléments visés par une règle
Classe	Attribut <code>class</code> du HTML, réutilisable, base du style
Cascade	Mécanisme d'arbitrage entre règles concurrentes
Spécificité	Score du sélecteur, décisif en cas de conflit
Héritage	Transmission automatique de certaines propriétés aux descendants
<code>inherit</code>	Valeur forçant la reprise de la propriété du parent
Modèle de boîte	Contenu, padding, border, margin
<code>box-sizing</code>	Détermine si <code>width</code> inclut ou non padding et border
Fusion des marges	Absorption de deux marges verticales adjacentes par la plus grande
Flexbox	Mise en page unidimensionnelle, en ligne ou en colonne
Axe principal	Direction de distribution : <code>justify-content</code>
Axe secondaire	Direction perpendiculaire : <code>align-items</code>
<code>gap</code>	Espace entre les éléments d'un conteneur flex ou grid
Pseudo-classe	Sélecteur d'état ou de position (<code>:hover</code> , <code>:nth-child</code>)
Contour de focus	Cadre signalant l'élément actif au clavier

Résumé de la semaine

Ce qu'il faut retenir

- Le CSS vit dans un **fichier séparé**, relié par `<link>` dans le `<head>`
- Le style passe par les **classes** ; l'`id` est réservé aux labels, aux ancrs et à JavaScript
- En cas de conflit : origine, puis **spécificité**, puis ordre d'écriture. `!important` n'est pas une solution
- Ce qui concerne le texte s'hérite, ce qui concerne la boîte ne s'hérite pas
- Toute boîte a quatre couches : contenu, padding, border, margin
- `*{ box-sizing: border-box; }` en première ligne rend les largeurs prévisibles
- Flexbox aligne sur **un** axe : `justify-content` sur l'axe principal, `align-items` sur l'axe secondaire — et tout s'inverse en `column`
- L'espacement entre éléments flex se fait avec `gap`
- Chaque `:hover` s'accompagne d'un `:focus-visible` ; on ne supprime jamais le contour de focus
- Les outils de développement disent quelle règle gagne, et pourquoi : la déclaration barrée a perdu

Fil rouge — MiniForum : le forum prend forme visuellement

Une feuille de style unique, partagée par les quatre pages, donne au MiniForum sa typographie, ses couleurs et ses espacements. La barre de navigation et les cartes de sujet sont construites en Flexbox.

C'est généralement la séance où le projet cesse d'être un exercice et devient « leur » site. Et le HTML des trois semaines précédentes n'a pratiquement pas été retouché : c'est précisément ce que permet une structure bien écrite.

Pour la semaine 5

- Le fichier `style.css` relié aux quatre pages et poussé sur GitHub
- Aucun attribut `style` et aucun `!important` dans le projet
- L'en-tête et la liste des sujets construits en Flexbox
- Le site reste parcourable au clavier, avec un contour de focus visible sur chaque élément interactif
- La semaine 5 ouvre le **responsive** : media queries, CSS Grid, unités relatives et variables CSS. Les couleurs choisies cette semaine deviendront des variables réutilisables jusqu'à la soutenance

Le détail pas à pas se trouve dans la fiche *S04_TP*.